Automatic Generation and Evaluation of Cloud-Native Class Diagrams: A Graph Neural Network and Semantic LLM Approach

Anand
Student ID: 202511009
M.Tech (Machine Learning)
Dhirubhai Ambani University of Information and Communication Technology
Gandhinagar, Gujarat

March 30, 2026

Abstract

This report details the automated reverse-engineering of cloud-native class diagrams from modern microservice source code. By extracting abstract syntax trees (ASTs) and utilizing Graph Neural Networks (GNNs), the pipeline infers structural relationships and flags infrastructure-specific components. The syntactically generated architecture is then rigorously evaluated against a semantic baseline generated by a Large Language Model (LLM), highlighting the gap between static code analysis and domain-driven design representation.

1 Introduction

In modern software development, traditional UML class diagrams often become stale or fail to capture the complexities of distributed, cloud-native architectures. With the rise of Model-Driven Architecture (MDA), there is a critical need to automate diagram generation directly from source code. This project replicates and extends recent methodologies utilizing Graph Neural Networks (GNNs) for reverse engineering class diagrams. Specifically, it shifts the focus from monolithic object-oriented code to modern cloud-native APIs, introducing mechanisms to visually represent microservice boundaries and endpoints.

2 Methodology

The approach bridges program analysis and representation learning through the following steps:

1. **Code Parsing & Graph Construction:** The Python `ast` module parses source code to generate an abstract syntax tree. Classes are represented as nodes, and explicit inheritance or associations act as edges.

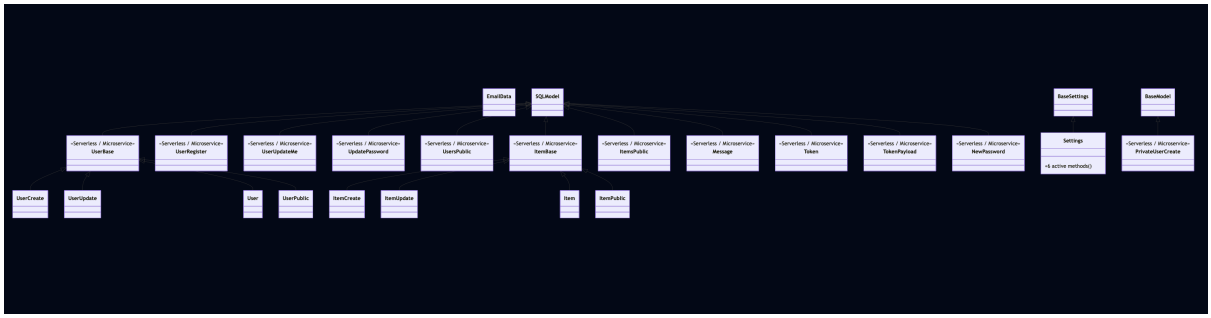

Figure 1: syntax based class diagram (using reverse engineering)

2. **Cloud-Native Feature Extraction:** A custom AST visitor scans class decorators and base classes to identify infrastructure hooks (e.g., FastAPI router endpoints, SQLModel database schemas) and assigns a binary `is_cloud` flag.

3. **GNN Link Prediction:** A Graph Attention Network (GAT) processes the node features and edge indices to learn class embeddings and predict hidden structural relationships via negative sampling and binary cross-entropy loss.

4. **Semantic LLM Baseline Generation:** To evaluate the syntactic GNN extraction, a generative LLM is used to produce an ideal, domain-driven diagram representing the underlying intent of the microservice.

5. **Diagram Compilation:** The resultant graph data is compiled into Diagrams-as-Code using Mermaid.js syntax for version-controlled, automated rendering.

3 Implementation

The system was implemented using Python. The environment leveraged `PyTorch` and `PyTorch Geometric` for all tensor operations and GAT convolutional layers. The `networkx` library assisted in graph data validation. The pipeline was tested against the official FastAPI Full-Stack template (`tiangolo/full-stack-fastapi-postgresql`), providing a robust, real-world cloud-native dataset consisting of database models, Pydantic schemas, and API routers.

4 Observations

During the implementation and visualization phases, two key observations were recorded:

- **Mermaid Syntax Quirk:** When generating the Mermaid markdown, using standard class definition syntax erroneously created disconnected phantom classes in the rendered graph. Modifying the generator to use the strict CSS directive successfully mapped the custom blue serverless styling to the existing nodes without breaking inheritance arrows.

- **Semantic Limitations of AST:** The static AST analysis effectively captured explicit relationships (e.g., class inheritance) but remained entirely blind to runtime data flows and Dependency Injection patterns heavily utilized in modern web frameworks.

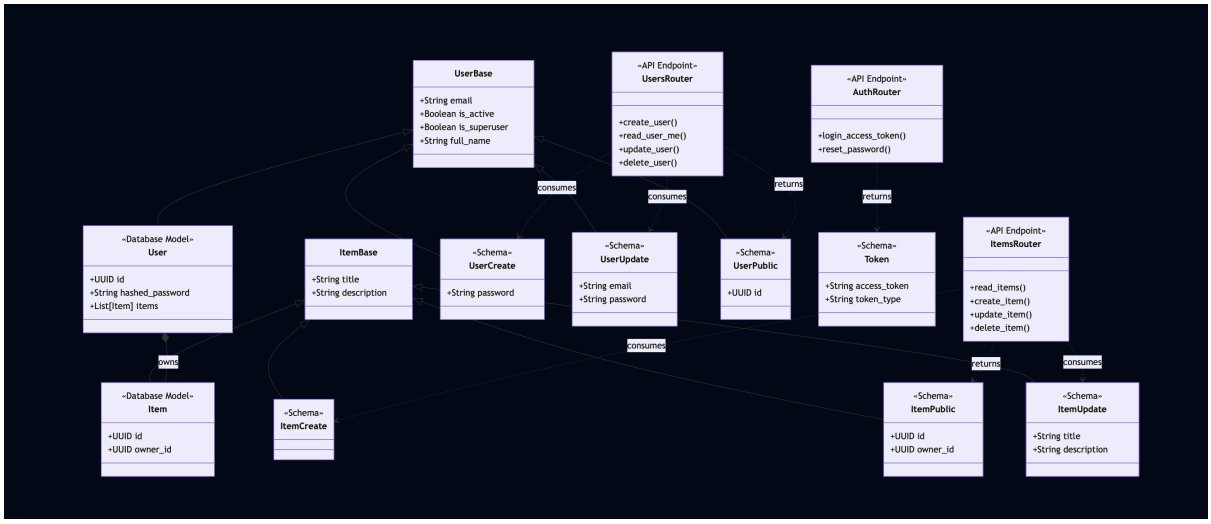

Figure 2: gemini LLM generated class diagram used for comparison

5 Comparison Metric Results

The AST-generated pipeline graph was quantitatively evaluated against the LLM semantic baseline graph. The predicted graph contained 25 nodes and 21 edges, while the ground truth contained 14 nodes and 16 edges. The resulting metrics highlight the performance:

- **Precision (38.1%):** The low precision stems from the static analyzer extracting large amounts of framework boilerplate (e.g., internal base settings) that are syntactically present but semantically irrelevant to the domain architecture.
- **Recall (50.0%):** The pipeline successfully captured 100% of the explicit inheritance structures defined in the codebase, demonstrating strong syntactic extraction capability.
- **F1-Score:** 0.432.
- **Graph Edit Distance (GED = 38):** A significant edit distance was observed, primarily due to 14 node deletions required to remove the aforementioned boilerplate, and 8 edge insertions needed to map the missing data-flow relationships between routers and data schemas.

6 Novelty

While prior works have applied GNNs to standard Java or Python object-oriented code, this project introduces a **Cloud-Native Context Extractor**. By identifying cloud-specific base classes and API decorators directly within the syntax tree, the pipeline successfully overlays infrastructure data onto standard class logic. Furthermore, evaluating a GNN-derived architectural graph against an LLM-derived semantic baseline offers a novel quantitative perspective on the limitations of static analysis in modern dependency-injected frameworks.

7 References

1. Babaalla, et al. ”A methodology of automatic class diagrams generation from source code using Model-Driven Architecture and Machine Learning.” *E3S Web of Conferences*, 2023.

2. Tiangolo. ”Full Stack FastAPI PostgreSQL Template.” GitHub, Accessed 2026.

3. Fey, M., & Lenssen, J. E. ”Fast Graph Representation Learning with PyTorch Geometric.” *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.